firmware\cube_v2.cpp

```
// ============================================================
// Prototype v3_2 - child Node
// ATtiny1616
//
// PlatformIO structure:
//   include/config.h
//   include/wire_bus.h
//   src/wire_bus.cpp
//   src/main.cpp
// ============================================================

#include <Arduino.h>
#include <avr/io.h>
#include <util/delay.h>
#include <stdint.h>
#include <stdbool.h>

#include "config.h"
#include "wire_bus.h"

// ------------------------------------------------------------
// Wire descriptors
// ------------------------------------------------------------

// Upstream orientation ports.
// Index in this array equals orientation value.
static const wire_t upstream[] = {
    { &PORTA, PIN1_bm, 1 },     // PA1 — orientation 0
    { &PORTA, PIN3_bm, 3 },     // PA3 — orientation 1
    { &PORTA, PIN4_bm, 4 },     // PA4 — orientation 2
    { &PORTA, PIN5_bm, 5 },     // PA5 — orientation 3
};

```

```
34 //suggestion: replace UPSTREAM_COUNT with upstream_count to avoid hardcoding the count
35 static const uint8_t upstream_count = sizeof(upstream) / sizeof(upstream[0]);
36
37 static const wire_t child_0 = { &PORTB, PIN0_bm, 0 };  // PB0 — child port 0
38 static const wire_t child_1 = { &PORTB, PIN1_bm, 1 };  // PB1 — child port 1
39
40 static const wire_t *children[CHILD_COUNT] = {
41     &child_0,
42     &child_1
43 };
44
45 // ------------------------------------------------------------
46 // Release all communication lines
47 // ------------------------------------------------------------
48
49 static void release_all_lines(void)
50 {
51     for (uint8_t i = 0; i < upstream_count; i++) {
52         wire_release(&upstream[i]);
53     }
54
55     for (uint8_t i = 0; i < CHILD_COUNT; i++) {
56         wire_release(children[i]);
57     }
58 }
59
60 // ------------------------------------------------------------
61 // Fast upstream scan
62 // scans all upstream lines for LOW and returns index of first detected LOW line (orientation value), or -1 if no line is LOW
63 // ------------------------------------------------------------
64
65 static int8_t scan_upstream(void)
66 {
67     for (uint8_t i = 0; i < upstream_count; i++) {
68         if (!is_line_high(&upstream[i])) {
69             return (int8_t)i;
```

```
        }
    }

    return -1;
}

// ---------------------------------------------------------
// Wait for upstream command
// ---------------------------------------------------------

static bool wait_for_valid_upstream_cmd(
    const wire_t **active_line,
    uint8_t *orientation_mask
)
{
    while (1) {
        int8_t ori = scan_upstream(); // detects orientation line - 0/1/2/3 or -1 if no low is detected

        if (ori < 0) {  //if no line is low, repeat orientation scan
            continue;
        }

        const wire_t *w = &upstream[ori]; //assignment (zuordnung) of detected orientation line/pin to w

        uint8_t b = 0;

        if (!rx_byte_data_frame(w, &b)) { //receiving of message (sent bits) without start bit, false if no valid bits are received within timeout
            continue;
        }

        //checks if received byte is CMD_STATUS, if not, repeat orientation scan and go into loop
        if (b != CMD_STATUS) {
            for (uint8_t i = 0; i < upstream_count; i++) { //release all upstream lines
                wire_release(&upstream[i]);
            }
```

```
105             wire_release(&child_0); //release child lines, just in case
106             wire_release(&child_1); //release child lines, just in case
107             continue;
108         }
109 
110         // if correct CMD_STATUS is received, ausgabeparameter aus funktion: assign detected line to active_line and orientation
    value to orientation and return true
111         *active_line = w;
112         *orientation_mask = (uint8_t)ori;
113 
114         return true;
115     }
116 }
117 
118 // ------------------------------------------------------------
119 // Poll one child port
120 // ------------------------------------------------------------
121 
122 static bool poll_child(
123     const wire_t *w,
124     uint8_t *child_payload,
125     uint8_t max_child_payload_len,
126     uint8_t *child_payload_len
127 )
128 {
129     uint8_t b = 0;
130 
131     /*
132      * 1. Send CMD_STATUS to child.
133      */
134     tx_byte(w, CMD_STATUS);
135 
136     /*
137      * 2. Switch direction / wait before receiving.
138      */
139     _delay_us(TURNAROUND_US);
```

```
140     // wire_release(w);  redundant
141
142     /*
143      * 3. Wait for ACK_BUSY.
144      */
145     if (!rx_byte(w, &b, CHILD_TIMEOUT_US)) {
146         return false;
147     }
148
149     if (b != ACK_BUSY) {
150         return false;
151     }
152
153     /*
154      * 4. ACK received, now receive full DATA_FRAME.
155      *    rx_frame validates SYNC, LEN and CHECKSUM.
156      *    It returns only the child payload.
157      */
158     if (!rx_frame(
159             w,
160             child_payload,
161             max_child_payload_len,
162             child_payload_len,
163             FRAME_TIMEOUT_US
164         )) {
165         return false;
166     }
167
168     return true;
169 }
170
171 // ------------------------------------------------------------
172 // Arduino setup
173 // ------------------------------------------------------------
174
175 void setup()
```

```
{
    release_all_lines();

    Serial.begin(9600);
    Serial.println("Prototype v3 recursive node started");
}

// ------------------------------------------------------------
// Main loop
// ------------------------------------------------------------

void loop()
{
    /*
     * These variables must be static because orientation detection
     * should only happen once, but the detected active upstream line
     * must still be available in later loop() iterations.
     */
    static bool orientation_known = false;
    static const wire_t *active_upstream = nullptr; //code merkt sich dauerhaft die Orientierungsleitung
    static uint8_t orientation_mask = 0x00; //orientation mask,now 1 byte, later combined with child_mask to one byte
    /// create variables (LEN_gesamt, LEN_Loop child etc.)

    /*
     * 1. Detect orientation only once.
     */
    if (!orientation_known) {
        wait_for_valid_upstream_cmd(&active_upstream, &orientation_mask); // selects active upstream line and orientation value
(orientation value must be transformed to orientation_byte for sending )
        orientation_known = true;
    }  else {
        // if orientation is already known, no need to check orientation just receive CMD_STATUS
        uint8_t b = 0;

        if (!rx_byte(active_upstream, &b, PARENT_TIMEOUT_US)) {
            Serial.println("Timeout: no byte received");
```

```
            return;
        }

        if (b != CMD_STATUS) {
            Serial.println("Unexpected byte received");
            return;
        }
    }
    /* you can also do create a own function instead:
        static bool wait_for_cmd_on_known_upstream(const wire_t *w)
    {
        uint8_t b = 0;

        if (!rx_byte(w, &b, PARENT_TIMEOUT_US)) {
            return false;
        }

        return (b == CMD_STATUS);
    }*/

    /*
     * Safety check.
     * Should normally never be true after orientation_known == true.
     */
    if (active_upstream == nullptr) {
        orientation_known = false;
        return;
    }

    /*
     * 2. Send ACK_BUSY upstream on the detected active line.
     */
    _delay_us(TURNAROUND_US);
    tx_byte(active_upstream, ACK_BUSY);
    //wire_release(active_upstream);

```

```
247      /*
248       * 3. Poll own children with retries.
249       *
250       * Child payloads are stored separately first.
251       * This keeps the final payload order correct:
252       * child 0 payload first, then child 1 payload.
253       */
254      uint8_t child_payloads[CHILD_COUNT][MAX_CHILD_PAYLOAD_LEN];   //temporary storage (2d array) for child payload results -> child_payloads[child_index][payload_index] = byte received from child
255      uint8_t child_payload_len[CHILD_COUNT] = {0};                 // temporary storage for child payload lengths -> child_payload_len[child_index] = length of payload received from child
256      bool child_done[CHILD_COUNT] = {false};                       // stores if child has already responded and can be skipped
257
258      // poll children with retries
259      for (uint8_t attempt = 0; attempt < CHILD_RETRIES; attempt++) { //loop for every retry
260          for (uint8_t i = 0; i < CHILD_COUNT; i++) { //loop for every child
261
262              if (child_done[i]) {   // if child[i] has already responded -> true -> skip
263                  continue;
264              }
265
266              //receives childs payload and payload length and stores it in temporary storages
267              if (poll_child(               //poll child[i]
268                      children[i],          //with wire/pin of child[i]
269                      child_payloads[i],    //and receive and store the payload in child_payloads[i]
270                      MAX_CHILD_PAYLOAD_LEN, //with max length of MAX_CHILD_PAYLOAD_LEN to avoid overflow
271                      &child_payload_len[i]  //and stores the received payload length from child[i]
272                  )) {
273                  child_done[i] = true;      //if everything correct sets child_done[i] to true -> this child will be skipped in next attempts
274              }
275          }
276      }
277
278      /*
279       * 4. Build own/nodes payload:
```

```
     *
     * Payload format:
     *   NODE_ID ORIENTATION CHILD_MASK CHILD_PAYLOADS...
     *
     * The parser later expects child payloads in child-port order.
     */
    uint8_t payload[MAX_PAYLOAD_LEN]; //final payload to be sent upstream
    uint8_t payload_len = 0;          //length of final payload
    uint8_t child_mask = 0x00;        //child_mask to be sent in final payload

    //stores own payload into the first payload position, ++ = incresas payload_len after writing, so next byte will be written on the next position
    payload[payload_len++] = NODE_ID; //writes NODE_ID into the first free payload position
    payload[payload_len++] = orientation_mask; //writes orientation mask byte into the next free payload position
    payload[payload_len++] = 0x00;   // placeholder for child_mask
    // e.g payload[0] = NODE_ID, payload[2]= 0x00; payload_len = 3

    //in a loop add children payloads to the final payload
    for (uint8_t i = 0; i < CHILD_COUNT; i++) {
        if (!child_done[i]) {  //if child[i] didn´t respond, skip it
            continue;
        }

        /*
         * Check whether this child payload still fits.
         * If not, do not set the child bit because the payload is not included.
         */
        if ((uint16_t)payload_len + child_payload_len[i] > MAX_PAYLOAD_LEN) {
            continue;
        }

        //create child_mask
        child_mask |= (1 << i);   //set bit for child[i]n in child_mask (LSB = child 0)

        //copy child payload into final payload
        for (uint8_t j = 0; j < child_payload_len[i]; j++) {
```

```
            payload[payload_len++] = child_payloads[i][j];  //copies stored payload into final payload and increases payload_len for next byte
        }
    }

    /*
     * Now write final child_mask into own node header.
     */
    payload[2] = child_mask;  //before was in payload[2] the placeholder 0x00

    /*
     * 5. Send complete frame upstream.
     */
    _delay_us(TURNAROUND_US);
    tx_frame(active_upstream, payload, payload_len);
    wire_release(active_upstream);

    /*
     * 6. Release all lines.
     */
    release_all_lines();

    /*
     * 7. Debug output.
     */
    Serial.print("Sent payload_len = ");
    Serial.println(payload_len);

    Serial.print("orientation = ");
    Serial.println(orientation_mask);

    Serial.print("child_mask = 0b");
    if ((child_mask & 0b10) == 0) {
        Serial.print("0");
    }
    Serial.println(child_mask, BIN);
```

```
350 }
```